

Forecasting county-level power outage ratios from archived ECMWF forecasts

Huawei Tech Arena 2026 — Topic 2, AIDC Power-Supply Risk Forecasting, Phase 1

Author: Tommaso Facchin (solo entry) **Tasks:** A (48 h / 1 h) and B (6 h / 15 min) **Test window:** 1 Sep – 30 Nov 2025
Date: 21 August 2026

1. What we built, and what came out of it

The system predicts $x = \text{customers_out} / \text{total_customers}$ for five US counties, at every lead time both tasks require, from archived ECMWF IFS HRES forecast runs plus the observed EAGLE-I outage history available at each issue time. It is a single LightGBM regressor with a Tweedie objective, one model covering both tasks, with lead time as an explicit feature, blended toward a persistence forecast by a weight fitted per lead bucket. The submitted file contains 65,880 rows: 92 daily Task A batches and 365 six-hourly Task B batches, five counties each, generated from 93 distinct archived forecast runs. It is built from 242 forecast runs across 102 counties — 1.75 million rows — spanning September 2024 to August 2025, of which the 1.24 million before 1 May 2025 are used for training and the rest held out.

The honest headline is that the learned model contributes far less than the way it is combined with the observed outage state. Against an always-zero forecast, which is a strong baseline when 69.0 % of the target values are exactly zero, the model alone reduces RMSE by 4.3 % on the forward-in-time holdout and MAE by 7.2 % on the rows that actually contain an outage. Two things dominate it: which county the row belongs to (47 % of total split gain) and the recent observed outage state (19 %). Weather contributes, but less than we assumed at the start of the week.

The result that changed the submission came last. Evaluated on a seasonally matched autumn window rather than on the late-spring holdout — Section 7.1 — the same model alone is worth only 1.8 % against always-zero, while blending it with persistence is worth **23.1 %**. Autumn outages are multi-day storm-restoration events, so the observed state at issue time stays informative across the entire 48-hour Task A horizon; late-spring outages are convective and decay within hours. The blend weights were originally fitted in the wrong season, and refitting them in the right one is the single largest improvement in this submission.

2. Choosing the five counties

The guidelines let each team pick its own five counties and ask for the choice to be justified. Our first instinct — rank all ~3,050 EAGLE-I counties by how often they report an outage and take the top five — is a trap, and it is worth describing because the trap is invisible in the output. Ranking by the fraction of 15-minute intervals with any nonzero `customers_out` returns Harris County TX, Miami-Dade, and their peers at rates of 99.9 %. Those counties are not unusually fragile. They are unusually large, and in a county with three million customers there is essentially always somebody off supply somewhere. What that ranking recovers is population, not weather risk.

So we ranked instead on the fraction of intervals with x above 1 %, which requires an event large enough to be visible at county scale. On top of that we required continuous EAGLE-I coverage across the window (a reporting gap and a quiet county look identical in a sparse event log), a denominator above 5,000 customers so that x is not dominated by single-household noise, and diversity of weather regime across the final five rather than five variations of the same climate.

The activity statistics come from January–August 2025 only, not the multi-year climatology we wanted: the forecast archive that supplies our features starts in March 2024 and the API budget did not stretch back further. That window at least stops before the evaluated one opens, so the selection never looks at a month it will be scored on.

Table 1 — The five reporting counties. Activity statistics cover 1 Jan – 31 Aug 2025 at 15-minute resolution, 23,328 intervals per county. “Significant” means $x > 0.01$.

FIPS	County	State	Customers	Significant	Max x	Regime, and why it is in the set
72013	Arecibo	Puerto Rico	41,122	43.5 %	1.000	Tropical/Caribbean. A chronically fragile grid and by a wide margin the strongest signal anywhere in the dataset.
37119	Mecklenburg	North Carolina	588,615	0.4 %	0.023	Piedmont/south-east. Hurricane remnants inland plus severe summer convection; Charlotte metro, so a large and well-reported denominator.

FIPS	County	State	Customers	Significant	Max x	Regime, and why it is in the set
54005	Boone	West Virginia	13,090	12.4 %	0.343	Appalachian interior. Ice storms and thunderstorm downbursts on wooded terrain; no maritime influence at all.
26097	Mackinac	Michigan	8,757	12.2 %	0.866	Great Lakes. Winter storms and repeated near-total local outages — the highest-variance county of the five.
22071	Orleans	Louisiana	212,936	0.9 %	0.245	Gulf Coast. Included deliberately despite a thin January–August signal: peak hurricane season falls inside the test window, and leaving the regime out because our proxy window misses it would be exactly the wrong inference.

Orleans is the one selection that a pure ranking would not have made, and we would defend it as the most important of the five. September to November is the Gulf hurricane season. A selection procedure calibrated on January to August will systematically under-rate Gulf counties, because their season has not started yet in the data it looks at. Correcting for that by hand is not cherry-picking; refusing to correct for it would be.

One caveat on this table that we would rather state than bury. The ranking ran before we found the denominator problem described in Section 3.4, so it used the `MCC.csv` customer counts throughout, and Mecklenburg's was wrong by a factor of 20.9. That inflated every x we computed for the county and carried it into the shortlist as the strongest mainland signal. Table 1 is recomputed on the corrected denominators, and on those numbers Mecklenburg is the *quietest* of the five, not the second loudest: its significant-event rate falls from 13.8 % to 0.4 %.

We kept it. The regime argument that put it on the shortlist is unaffected by the arithmetic error, and swapping it now would cost roughly 185 archived forecast calls — 93 to regenerate the submission at new coordinates and about 92 to give the replacement county training coverage — which at the observed daily quota is two of the three days left. Those calls are better spent on the seasonal training data described in Section 8. A team with more API budget should re-run the selection on corrected denominators; the selection code takes the reconciled table as its input already.

3. Data acquisition and preprocessing

3.1 Ground truth: finding EAGLE-I, and then reading it correctly

The competition materials point at the ORNL/OSTI record, which routes to a Globus endpoint covering 2014–2022 and requiring an account. We lost most of a day believing that was the only scriptable route. It is not: the complete 2014–2025 release, 17 files and about 11.6 GB, sits on figshare as article 24237376 (version 4, February 2026), and every annual file is an anonymous HTTPS GET against `ndownloader.figshare.com`. Our downloader resolves that file table from the figshare API rather than hard-coding it, and fetches whichever years are asked for.

The annual files do not share a schema, and the drift is silent rather than fatal. Files from 2014–2022 and 2025 use `fips_code`, `county`, `state`, `customers_out`, `run_start_time`. The 2023 file calls the fourth column `sum`. The 2024 file adds a sixth column, `total_customers`. A straightforward `pd.concat` across years therefore produces a `customers_out` column that is entirely null for 2023 without raising anything at all. Our loader normalizes headers through an explicit alias table before concatenating, and raises on an unrecognized rename instead of degrading into nulls.

3.2 From sparse events to a dense target

EAGLE-I emits a row only when `customers_out > 0`. Everything else is implicitly zero, and a model trained only on rows that exist would never see the negative class it mostly has to predict. Preprocessing reindexes each county onto a complete 15-minute grid across the period of interest and zero-fills the gaps, then computes x and clips it to $[0, 1]$. The clip is not cosmetic: a handful of source rows report more customers out than the county's modeled customer count, which is a known mismatch between the outage feed and the customer-count file rather than a corrupt row to discard.

About 6.5 % of raw EAGLE-I rows carry an explicit null `customers_out`. That is not the same as an absent row. An absent row means no outage; an explicit null means the upstream ETL could not get a reading. Coercing nulls to zero would encode “we do not know” as “confirmed no outage”, and would bias the model toward under-predicting risk during exactly the periods when utility reporting is itself degraded — which is plausibly correlated with the severe weather we are trying to predict. We treat the two cases differently: a null in the *target* means the row is dropped, since there is nothing to train or score against; a null in an *autoregressive input* is left as a null and handed to LightGBM, which learns a default split direction for missing values. Imputing a fabricated number there would be strictly worse.

3.3 Weather: the Single Runs archive and its real rate limits

The guidelines require that prediction-time weather be the forecast genuinely available at the issue time, which rules out the Historical Forecast API (it stitches together only the first hours of successive runs and so does not preserve lead-time-specific values) and rules out observed reanalysis. We use Open-Meteo's Single Runs API with `models=ecmwf_ifs`, which returns a specific archived initialization of IFS HRES at 9 km. That archive begins on 14 March 2024; every other model in the endpoint is archived only from April 2026 and cannot overlap the test window. So IFS HRES is not a preference here, it is the only option, and its start date is the hard boundary on how far back training data can go.

The binding constraint on this project turned out to be the endpoint's rate limits. The published free-tier terms allow 10,000 calls per day, 5,000 per hour and 600 per minute. The Single Runs endpoint enforces three much tighter windows, none of them documented, and we discovered each one only by hitting it: a per-minute cap, a per-hour cap that needs roughly 62 minutes to clear, and a daily cap that in practice cut in somewhere around 110 to 130 calls. We found the daily one on the night of 19 August, mid-download, and it changed the shape of the project, because the same quota also pays for generating the submission. Two responses followed, both of which cost accuracy:

- The training sample was cut to 102 counties, stratified by Census region and by activity tercile with a cap of three counties per state per cell (Texas alone has 151 eligible counties, and an unstratified draw would have produced a Texas weather model). Runs were taken at 00Z and 12Z on every second day rather than all four runs daily.
- Task B shares Task A's forecast run rather than fetching a fresher one every six hours. Fetching per-issue-time would have cost 367 distinct calls for the test window; sharing one run per calendar day costs 93. The forecast a Task B batch consumes is therefore 12 to 30 hours old rather than 6, which is still a forecast genuinely available at issue time and so still compliant, but it is a real skill concession made for a quota reason. Reverting it is a one-line change if the quota stops being scarce.

Every response is cached to a local SQLite store keyed on the request URL, so a given (coordinates, run, variables) request is fetched exactly once in the life of the project and every re-run of the pipeline is free. That cache is what makes retraining and regenerating the submission cost nothing, and it is why the download scripts are safe to kill and restart.

3.4 The denominator, which is the biggest uncontrolled risk in the submission

x is a ratio, so the customer count in its denominator multiplies every number we predict. The organizers maintain their own fixed reference value and do not publish it. We therefore have to pick a denominator and say clearly which one we picked.

The natural choice is `MCC.csv`, the modeled county customer file shipped with the EAGLE-I release. On 20 August we checked it, summing it per state against `coverage_history.csv`, an independent file from the same release that gives each state's total. MCC reproduces the state total to within a few percent almost everywhere. For North Carolina it lands at 1.82 M against a published 5.30 M, missing about two thirds of the state. Mecklenburg County, one of our five, carries 28,172 customers in MCC against 588,615 in EAGLE-I's own 2024 annual file. Every x we had computed for Mecklenburg, in the training target and in the predictions, was inflated by a factor of 20.9.

The obvious fix — switch to the 2024 file's column — is wrong in the other direction elsewhere. Puerto Rico's state total agrees almost exactly between the two sources (1,485,262 against 1,485,249), but they allocate it differently across municipios: Arecibo is 41,122 in MCC and 191,803 in the 2024 file, against a resident population of roughly 87,000. The 2024 figure implies more electricity customers than people. Neither source is uniformly right, so a blanket swap trades a broken North Carolina for a broken Puerto Rico.

The rule we adopted is per state, not per county: for each state, take the county allocation from whichever candidate source reproduces that state's independently published total more closely, with ties going to MCC as the documented default. Per state, because the failure mode we observed is a whole-state one, and because a per-county rule would have no independent quantity to check itself against. Three states end up using the 2024 in-file allocation (Kansas, North Carolina, Texas) and the other 49 keep MCC. The full table is in `data/processed/denominator_reconciliation.md`. This still cannot be validated against the organizers' reference, which is precisely why it is written down here.

3.5 Timestamp alignment and spatial matching

Everything internal is naive UTC. EAGLE-I's `run_start_time` is parsed as UTC and never localized; the Open-Meteo response comes back timezone-aware and is converted to naive UTC at the point where the weather table is renamed for joining, because a tz-aware and a tz-naive column merge into an empty result rather than an error. Output timestamps are formatted as ISO 8601 with an explicit Z. FIPS codes are strings everywhere, zero-padded to five characters at every load, which matters for Puerto Rico and for every state whose code starts with a zero.

Spatial matching is the simplest thing that is defensible: each county is represented by its internal-point centroid from the 2025 US Census Gazetteer, and the forecast is requested at that single coordinate. Phase 1 targets counties rather than sites, so there is no site-to-county assignment to make; the approximation is that one point represents a county's weather, which for a lake-shore county like Mackinac is a real simplification. Area-averaging over a grid of points per county would be better and costs a multiple of the API budget we did not have.

4. Features

Fifty features in five groups. The design principle throughout is that a feature must be a function only of data timestamped at or before the issue time, checked by a single shared assertion (`code/features/causality.py`) rather than by scattered ad-hoc guards.

Raw forecast fields (12): 10 m wind speed, gusts and direction; precipitation, rain and snowfall; 2 m temperature and dew point; surface pressure; CAPE; cloud cover; freezing-level height.

Derived weather (11). Gust cubed, because wind loading on a conductor or a tree scales roughly with the cube of speed and the linear gust value compresses exactly the range that breaks things. Centered rolling gust maxima over ± 3 h, ± 6 h and ± 12 h, which answer “is this hour inside a windstorm” rather than “is this hour windy”. Cumulative precipitation over the preceding 6 and 24 hours, as a proxy for soil saturation, since saturated ground plus gusts is what uproots trees. A wind $\times$ rain interaction term. Three-hour pressure change and one-hour gust jump for rate-of-change. All of these are computed strictly inside a single forecast trajectory, which is why a centered window is causally fine: those hours were disseminated together in one run, and causality here is run time against issue time, not neighboring hours within a forecast that has already arrived.

Climatological exceedance (12). Per-county p95, p99 and p99.9 of gusts and precipitation, plus boolean flags for exceeding each. These are what let one model serve both a Gulf county and an Appalachian one: 60 km/h means something different in each, and the exceedance flag says how unusual the value is locally instead of how large it is absolutely. The thresholds are fitted once on the training portion of the split only, saved with the model, and reapplied unchanged at prediction time. Recomputing them at serving time is a subtle and destructive bug: a single 62-hour forecast batch has a p95 gust roughly half the training p95 (26.4 against 54.4 km/h for Orleans), so the exceedance features would quietly mean something different in production than in training.

Autoregressive outage state (11). Observed x at issue time and at -15 min, -30 min, -1 h, -2 h and -6 h; one- and two-hour trends; the 24-hour rolling maximum; a flag for an outage in progress and the length of the current run. The guidelines restrict only the *weather* input to what a forecast could have provided; observed outage history up to issue time is realistic, since EAGLE-I and the ODIN feed are near-real-time, and it is the main source of genuine sub-hourly skill for Task B. IFS has no native 15-minute output, so the weather a Task B row sees is interpolated from hourly values and carries no true sub-hourly information. Whatever timing skill Task B has comes from this group.

Temporal and static (6). Hour of day and day of year as sine/cosine pairs, lead time in hours, county FIPS as a categorical, and the customer count.

5. Model and training

One LightGBM regressor with `objective=tweedie` and `tweedie_variance_power=1.5`, 500 trees, learning rate 0.05, 63 leaves, minimum 50 samples per leaf, 0.8 row and column subsampling. The Tweedie family is the natural fit for a non-negative continuous target with a point mass at zero, which is what x is; it handles the zero inflation inside the objective rather than by gluing two models together. Both tasks are served by the same model with lead time as a feature, since a per-task split would halve the training data for no structural reason.

The split is temporal and never random: training on rows before 1 May 2025, validating on everything after. A random split would put two rows from the same storm fifteen minutes apart on opposite sides and make every validation number meaningless. The held-out period is not an unrepresentatively calm stretch: 36.0 % of its rows are positive against 29.0 % in training. Because the weather download kept running to the end, every run it added after 1 May lands in the holdout rather than in training — the evaluation set is 502,777 rows against 1.24 million trained on.

Two decisions came out of bugs rather than design. There is no early stopping: an earlier version registered no validation set at all through a keyword-name error, stopped after 25 of 500 trees, and deployed a stump with a maximum prediction of 0.012 against 0.086 — no dynamic range, and unable to signal a severe event. We fixed it by removing early stopping rather than repairing the wiring, because stopping on the validation set and then reporting metrics on it would make every number here optimistic. And the model is saved as a bundle together with the fitted state its features depend on (the FIPS category ordering and the climatology thresholds), because re-deriving either at prediction time produces a model that silently answers a different question.

5.1 Blending with persistence, and the season it is fitted in

The lead-time curve showed persistence beating the model below about twelve hours and losing badly beyond it, so a single predictor is the wrong shape for a submission whose two tasks sit on opposite sides of that crossover. There is also a specific train/serve mismatch behind it. During training, issue time and run time are the same instant, so the model's `lead_hours` feature means both “how far ahead is the target” and “how stale are the autoregressive features”. At serving time the two come apart: a Task B row asks for a fifteen-minute-ahead prediction while carrying `lead_hours` between 12 and 36, because that is the age of the forecast. The model learned to discount autoregressive features at long lead, and nothing in its input says that this time they are minutes old. Persistence has no such confusion.

So we fit a weight per operational-lead bucket that minimizes RMSE between the model and a plain persistence forecast, and apply it at prediction time on the operational lead. RMSE rather than MAE, because on a target that is 69.8 % exact zeros, MAE is minimized by collapsing toward zero and a blend tuned on it would simply pick whichever component predicts less.

Which held-out period the weights are fitted on turned out to matter far more than the blend itself. Fitted on the forward-in-time split, which lands in May to August, the weights come out at 0.55 / 0.35 / 0.15 / 0.10 / 0.05 across the five lead buckets: persistence gets about half the prediction at the shortest lead and a twentieth of it beyond two days. Fitted by the identical procedure on the autumn holdout described in Section 7.1, they come out at **0.90 / 0.80 / 0.75 / 0.60 / 0.45** — persistence is worth more at every lead, and still worth nearly half the prediction two to three days out. The reason is physical rather than statistical: late-spring outages are short convective events that decay within hours, so yesterday's state says little about tomorrow, whereas autumn outages are storm-restoration events lasting days.

The two weight sets are worth very different amounts in the season that will be scored. Measured on the autumn window, where always-zero scores RMSE 0.036474 and the model alone 0.035833 (−1.8 %), the spring-fitted weights reach 0.032648 (−10.5 % against always-zero) and the autumn-fitted weights 0.028035 (−23.1 %). Back on the May–August split the ordering reverses and the autumn weights give up 6.3 points of skill — in a season the submission is never scored in. We therefore ship the autumn-fitted weights. They are fitted against a model retrained on 2025 alone, never against the deployed model: the deployed model was trained on the autumn rows, so its predictions there are in-sample and would have understated exactly what this measures.

One caveat we would rather state than bury: autumn 2024 contains Helene and Milton, so these weights would be too persistence-heavy for an unusually quiet autumn. The downside is bounded — during calm periods `x_at_issue` is approximately zero, so a heavy persistence weight shrinks predictions toward zero, the direction this target rewards anyway.

6. Handling the class imbalance

In the training table 69.0 % of rows have `x` exactly zero, 31.0 % are nonzero, and only 6.6 % exceed 0.001. The mean is 0.00218. Any error metric averaged over all rows is therefore a statement about the zeros, and an always-zero forecast is a genuinely strong competitor on MAE rather than a strawman. We address this in four places.

The objective absorbs the zero mass directly: Tweedie with a variance power of 1.5 is a compound Poisson-gamma, which is a distribution with an atom at zero and a continuous positive part, so the model is not being asked to fit a point mass with a Gaussian.

Evaluation is stratified rather than pooled. Every result in Section 7 is reported both over all rows and restricted to rows where the observed `x` exceeds 0.001, and the always-zero baseline appears in every table so that a reader can see how much of a number is skill and how much is arithmetic on zeros. We also report maximum prediction per model, because a model that looks competitive on average while never predicting above 1 % of customers out is useless for the actual application.

The hurdle formulation in Section 7 attacks the imbalance structurally by splitting the question in two: a classifier for whether an event happens at all, and a severity regressor trained only on positives, so that the severity head never sees a zero and is not dragged toward one.

We planned a fourth measure, undersampling quiet periods with inverse-probability weights, and did not need it: the forecast archive already constrains the table to 242 run times rather than a row every fifteen minutes, so at 1.75M rows it still trains in half a minute, and undersampling would have traded information away to buy speed we already had.

7. Results

Unless stated otherwise, numbers below are on the forward-in-time held-out period after 1 May 2025 (502,777 rows, of which 40,513 have observed x above 0.001), using a model that never saw a row from that period, with thresholds and category encodings fitted only on the earlier portion. Section 7.1 then repeats the comparison on a seasonally matched autumn window, which is the one we would trust for the test period.

Table 2 — Baselines, model, and feature-group ablations. Each ablation is a full retrain with that group removed, so its effect is marginal given everything else, not a univariate correlation. Lower is better in the first three columns.

Predictor	MAE	RMSE	MAE, events	Max pred.	Δ RMSE vs full
always zero	0.001524	0.013900	0.018294	0.000	+4.44 %
persistence (x at issue time)	0.002124	0.015878	0.019335	0.415	+19.31 %
per-county climatology	0.003178	0.014276	0.018163	0.040	+7.27 %
full model (LightGBM Tweedie)	0.001629	0.013309	0.016970	0.121	—
– raw forecast fields	0.001643	0.013345	0.017085	0.123	+0.28 %
– derived weather	0.001645	0.013382	0.016897	0.074	+0.55 %
– climatological exceedance	0.001650	0.013400	0.016766	0.096	+0.69 %
– autoregressive state	0.001761	0.013387	0.017255	0.155	+0.59 %
– temporal encodings	0.001732	0.013314	0.016933	0.126	+0.04 %
– static county features	0.001613	0.013334	0.016685	0.098	+0.19 %
hurdle, $\tau = 0$	0.001455	0.013391	0.016731	0.205	+0.62 %
hurdle, $\tau = 0.001$	0.001584	0.013429	0.016506	0.075	+0.91 %

The full model beats all three baselines on RMSE and on event-restricted MAE, and loses to always-zero on pooled MAE by a hair. That last point is not a defect, it is what pooled MAE measures on a target that is seven-tenths zeros: predicting a small positive number everywhere costs MAE on 322,016 zero rows to buy accuracy on 40,513 event rows. Per-county climatology is the strongest of the naive baselines on RMSE, which is itself informative — a large part of what any model can learn here is simply that some counties break more often than others.

The ablations are modest across the board and we are not going to dress them up. Every group now earns its place, but none of them earns much: climatological exceedance costs the most to remove (+0.69 % RMSE), then autoregressive state (+0.59 %), derived weather (+0.55 %), the raw forecast fields (+0.28 %), static county features (+0.19 %) and the temporal encodings (+0.04 %). That all six are positive is itself a result of adding the autumn runs: on the January–June table the twelve raw forecast fields were worth nothing measurable at all and two groups actively hurt. Feature importance tells the same story from the other side: county identity alone accounts for 47 % of total split gain, the autoregressive group for 19 %, and all weather together for 21 %.

Table 3 — The seven highest-gain features, as a share of the model's total split gain.

Feature	Gain share	Group
fips_code	47.4 %	static
x_at_issue	8.1 %	autoregressive
doy_sin	6.9 %	temporal
precip_cum_24h	6.3 %	derived weather
x_lag_2h	3.8 %	autoregressive
doy_cos	3.4 %	temporal
x_max_24h	2.5 %	autoregressive

Two readings were available when the training window held no hurricane at all: either county-level outage rates are mostly a property of the utility and the vegetation, or the training set simply contained too little severe weather for a weather response to be learnable. Adding the September–November 2024 runs — which do contain Helene and Milton — discriminates between them, and the answer is partly the second. Weather's share of split gain roughly doubled and every weather ablation turned positive. It did not, however, transform the model: county identity still dominates, and the features that carry weather's weight are the ones describing a sustained event (24-hour cumulative precipitation, 12-hour rolling gust maximum) rather than an instantaneous value.

7.1 Skill against lead time

The forward-in-time split above lands in May to August. The window being scored is September to November, and the two are not interchangeable: autumn RMSE on this target is more than twice the late-spring figure, because autumn outages are larger. So we re-ran the same comparison on a seasonally matched window — train on 2025, evaluate on September–November 2024, which the deployed model has seen but this probe's model has not. Time runs backwards in that arrangement, which is fine for a seasonal-transfer question and is used for no model selection: it measures how a model fitted outside autumn behaves inside it.

Table 4 — The same predictors in two seasons. Skill is RMSE reduction against always-zero within that season. Persistence is scored where x at issue time is defined (all autumn rows; 501,160 of 502,777 reference rows).

Predictor	Autumn RMSE	skill	Reference RMSE	skill
always zero	0.036474	—	0.013900	—
persistence alone	0.031443	+13.8 %	0.015892	−14.3 %
model alone	0.035833	+1.8 %	0.013309	+4.3 %
blend, spring-fitted weights	0.032648	+10.5 %	0.013139	+5.5 %
blend, autumn-fitted weights (submitted)	0.028035	+23.1 %	0.014014	−0.8 %

Read the two skill columns against each other and the case for the submitted configuration is the whole of it. Every predictor changes rank between seasons. Persistence swings from the best non-trivial forecast available (+13.8 %) to substantially worse than no forecast at all (−14.3 %), and the submitted configuration follows it down to −0.8 % out of season. The learned model is the steadiest thing in the table and also the smallest: between +1.8 % and +4.3 % wherever it is measured.

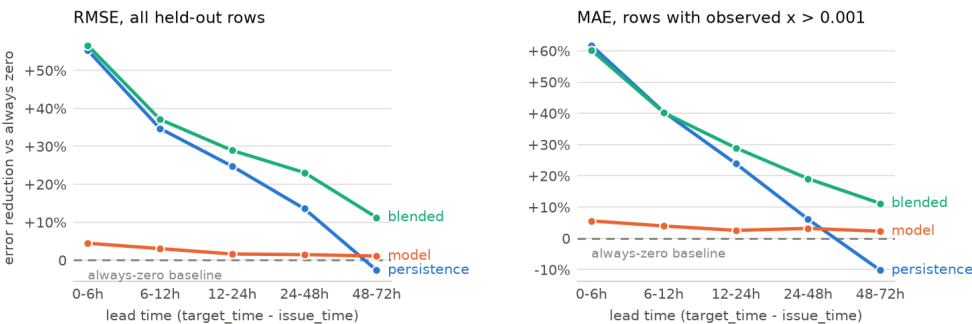


Figure 1. Error reduction against always-zero per lead-time bucket, on the 709,716 rows of the seasonally matched autumn window. Raw error per bucket is not comparable across buckets on this target — events are rare and clustered, so each bucket catches a different set of storms — which is why this is plotted as a skill score.

Figure 1 is the curve the guidelines say scoring will look at, in the season it will be scored in. Persistence decays from +55 % at 0–6 h to slightly negative past two days, which is the behaviour a persistence forecast should have. The model is nearly flat by comparison, holding a few percent at every horizon. The blend is above both at every bucket, not between them: it keeps most of persistence's short-lead advantage and still returns +11 % at 48–72 h, where persistence alone has gone negative. That is the shape we want for a submission whose Task B lives entirely in the first bucket and whose Task A runs out to the fourth.

7.2 The hurdle alternative

The hurdle model is the guidelines' class-imbalance question posed as an experiment: instead of one Tweedie head, a binary classifier for $P(x > \tau)$ multiplied by a severity head trained only on rows above τ . It splits the decision: with $\tau = 0$ it produces the best pooled MAE of anything we tried (0.001455, beating even always-zero) and with $\tau = 0.001$ the best event MAE (0.016506 against Tweedie's 0.016970), and it reaches a maximum prediction of 0.205 against Tweedie's 0.121 — more of the dynamic range a severe-event forecast needs. Both lose to Tweedie on RMSE, by 0.62 % and 0.91 %.

We submit Tweedie, having checked the choice in the season that matters rather than settling it on the reference split: in the autumn window the hurdle is behind on RMSE there too (0.036007 against 0.035833, and 0.028063 against 0.028035 after blending, the same weights coming out of the fit for both heads). The hurdle buys MAE and costs RMSE, consistently, at the price of serving two models instead of one. With the scoring metric unpublished we prefer the simpler head.

The severity head regresses $\log x$ under squared loss rather than x under a gamma objective. Gamma was tried first and produced predictions around 3.4×10^4 for a quantity bounded by 1: its log link puts almost no curvature on labels

spanning $1e-7$ to 1 and lets leaf values run away. Working in log space bounds the quantity by construction, at the cost of $\exp(E[\log x])$ being a geometric mean sitting below $E[x]$.

8. What limits these numbers

The training set is the limitation that matters, and it shrank at every stage of the week for reasons that were each individually defensible. A multi-year window became eighteen months once the IFS archive's March 2024 start was confirmed; the per-hour rate limit then cut the county sample from $\sim 3,050$ to 102 and the run cadence to every other day; the daily quota, found last, capped how many runs we could add. The table behind these numbers holds 242 forecast runs across 102 counties in 40 states, spanning 1 September 2024 to 4 August 2025; the model is trained on the 1.24 million rows of it that precede 1 May 2025.

The most obvious weakness in the earlier version of this submission was that its training window contained no Atlantic hurricane season while the evaluated window is mostly hurricane season. That is now closed: the September–November 2024 runs are in, which brings Helene and Milton with them, and the effect is visible in Section 7 — every weather ablation turned positive and weather's share of split gain roughly doubled. What the autumn data mainly bought, though, was not a better model. It was the ability to *measure* in the right season, and that measurement is what produced the blend-weight correction worth 23 % RMSE.

Which leaves an honest statement of what the submitted predictor is. The persistence weight runs from 0.90 at the shortest leads to 0.45 at three days, which works out to 0.83 averaged over the submitted rows — Task B is two thirds of the file and sits entirely in the shortest bucket. So most of what we submit is the observed outage state carried forward, and a minority of it is the learned weather model. We would rather report that than imply the ratio is the other way round. The learned part is real — it is what keeps the blend above persistence at long lead, where persistence alone is worse than predicting nothing — but it is the smaller contribution.

Four smaller caveats. The blend weights are fitted on a single autumn containing two major landfalling systems and would be too persistence-heavy for an unusually quiet one (Section 5.1 bounds the downside). The climatological quantiles are fitted on months rather than decades, so “p99 gust” here means “p99 of what this county saw in the training window”. Task B's weather is interpolated from hourly IFS output and contains no genuine sub-hourly information. And the denominator remains the largest uncontrolled risk in the submission: we predict a ratio whose divisor the organizers hold privately, and if their reference for a county differs from ours, our predictions for it are scaled wrong by exactly that factor however good the model is.

9. Data sources and licenses

Every source used, with the license verified at the source on 21 August 2026 rather than assumed.

Table 5 — Data sources and licenses.

Source	Use	License
EAGLE-I Recorded Electricity Outages 2014–2025 (Oak Ridge National Laboratory). Brelsford, Tennille, Myers, Chinthavali, Tansakul, Denman et al., figshare article 24237376 v4, 25 Feb 2026. doi:10.6084/m9.figshare.24237376.v4	Ground-truth outage counts; MCC.csv and coverage_history.csv for the denominator.	CC BY 4.0 , as returned by the figshare API for that article.
Open-Meteo Single Runs API single-runs-api.open-meteo.com	Delivery of archived ECMWF IFS HRES initializations, the sole weather input for both training and prediction.	CC BY 4.0 under the free non-commercial tier, per Open-Meteo's Terms of Use. Attribution: <i>Weather data by Open-Meteo.com</i> .
ECMWF IFS HRES open forecast data (originating model behind the above)	The forecast fields themselves.	CC BY 4.0 plus the ECMWF Terms of Use: “a subset of ECMWF real-time forecast data from the IFS and AIFS models is made available to the public free of charge ... governed by the Creative Commons CC-BY-4.0 licence”, subject to attribution.
US Census Bureau 2025 County Gazetteer 2025_Gaz_counties_national.txt	County internal-point centroids, the forecast coordinate for each county.	US Government work, not subject to domestic copyright (17 U.S.C. §105) — public domain.

Code, execution order and expected runtimes are in README.md. predictions.csv is regenerated end to end by python code/predict.py, which validates the file it writes.